

DPDP COMPLIANCE AUDIT

medusa

Digital Personal Data Protection Act 2023



Grade C — Needs Work

Scanned: 08 April 2026

Files indexed: 8446

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

This codebase has 6 critical compliance gap(s) that require immediate attention under the Digital Personal Data Protection Act 2023. An additional 10 significant gap(s) were identified that should be addressed within 30 days. The estimated total remediation effort is 5.2–19.8 engineering days.

Total findings: 25

HIGH: 6

MEDIUM: 10

INFO: 3

PASS: 2

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	22/22 (100%)	PASS
Section 6(6)	8	8/8 (100%)	PASS
Section 8(1)	18	0/18 (0%)	FAIL
Section 8	10	0/10 (0%)	FAIL
Section 8(3)	8	0/8 (0%)	FAIL
Section 8(4)	8	4.0/8 (50%)	WARN
Section 8(6)	8	5.6/8 (70%)	WARN
Section 9	6	6/6 (100%)	PASS
Section 11	6	6/6 (100%)	PASS
Section 16	4	2.0/4 (50%)	WARN
Section 5	2	1.0/2 (50%)	WARN
Section 13 — Grievance Redressal	4	4/4 (100%)	PASS

Estimated Total Remediation Effort: 5.2–19.8 engineering days

Data Flow Analysis

The following PII flow signals were inferred via static analysis (import/call adjacency). These are heuristic code-path chains, not proven runtime traces. Use them to guide developer review (confirm actual request → handler → service → database paths in your app).

Flow 1: Application Logs

PII fields: email

draft-order.ts

Flow 2: Application Logs

PII fields: email

user.ts

Flow 3: Application Logs

PII fields: password

create.ts

Flow 4: Application Logs

PII fields: password

create-db.ts → log-message.ts

Flow 5: Application Logs

PII fields: address

stock-location.ts

Rule	Section	Severity	File
DEBUG_MODE_ENABLED	Section 8(1) — Security	HIGH	packages/medusa/src/commands/develop.ts
CROSS_BORDER_TRANSFER_RISK	Section 16 — Cross-Border Transfer	MEDIUM	packages/admin/dashboard/src/routes/products/product-import/helpers/import-template
PASSWORD_NOT_HASHED	Section 8(1) — Security	MEDIUM	MULTIPLE
HARDCODED_SECRET	Section 8(1) — Security	HIGH	MULTIPLE
PLAINTEXT_PII_IN_LOGS	Section 8(1) — Security	HIGH	packages/design-system/toolbox/src/commands/tokens/command.ts
PLAINTEXT_PII_IN_LOGS	Section 8(1) — Security	HIGH	packages/medusa/src/migration-scripts/create-super-admin-role.ts
AUDIT_TRAIL_PARTIAL	Section 8(4) — Audit Trail	MEDIUM	N/A
RETENTION_PARTIAL	Section 8(3) — Retention	MEDIUM	N/A
NO_ERROR_HANDLING_IN_AUTH	Section 8(6) — Breach	MEDIUM	integration-tests/modules/helpers/create-authenticated-customer.ts
INCOMPLETE_DELETION_COVERAGE	Section 8 — Data Principal Rights	MEDIUM	N/A
THIRD_PARTY_PII_SHARING	Section 5 — Notice	MEDIUM	MULTIPLE
RETENTION_MISSING	Section 8(3) — Retention	MEDIUM	packages/medusa/src/api/admin/views/[entity]/columns/helpers.ts
RETENTION_MISSING	Section 8(3) — Retention	MEDIUM	packages/plugins/draft-order/src/admin/lib/schemas/address.ts
INTERNAL_ROUTE_PII_ACCESS	Section 7 — Legitimate Use	INFO	packages/core/js-sdk/src/admin/stock-location.ts
INTERNAL_ROUTE_PII_ACCESS	Section 7 — Legitimate Use	INFO	packages/cli/medusa-cli/src/commands/new.ts

INTERNAL_ROUTE_PII_ACCESS	Section 7 — Legitimate Use	INFO	integration-tests/modules/helpers/create-authenticated-customer.ts
CONSENT_WITHDRAWAL_PRESENT	Section 6(6) — Consent Withdrawal	PASS	packages/admin/dashboard/src/routes/workflow-executions/workflow-execution-detail
BREACH_INDICATORS_ADEQUATE	Section 8(6) — Breach	PASS	N/A
DEEP_REVIEW_VALIDATED	Section 8(1)	HIGH	integration-tests/http/__fixtures__/auth/medusa-config.js
DEEP_REVIEW_VALIDATED	Section 8(1)	MEDIUM	integration-tests/modules/helpers/create-authenticated-customer.ts
DEEP_REVIEW_VALIDATED	Section 8(1)	LOW	integration-tests/http/__fixtures__/auth/medusa-config.js
DEEP_REVIEW_VALIDATED	Section 8(1)	HIGH	integration-tests/api/medusa-config.js
DEEP_REVIEW_VALIDATED	Section 8(1)	LOW	integration-tests/setup-env.js
DEEP_REVIEW_VALIDATED	Section 8(1)	LOW	integration-tests/environment-helpers/setup-server.js
DEEP_REVIEW_VALIDATED	Section 8(1)	LOW	packages/admin/dashboard/src/routes/login/components/cloud-auth-login.tsx

1 finding(s) show rule-engine fallback text only (LLM enrichment cap or parse failure).

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (12–47 hours)

Fix immediately — these represent the highest legal exposure.

- Debug Mode Enabled — commands/develop.ts (4–16 hrs)
- Hardcoded Secret — MULTIPLE (2–9 hrs)
- Plaintext Pii In Logs — tokens/command.ts (1–3 hrs)
- Plaintext Pii In Logs — migration-scripts/create-super-admin-role.ts (1–3 hrs)
- Deep Review Validated — auth/medusa-config.js (2–8 hrs)

- Deep Review Validated — api/medusa-config.js (2–8 hrs)

Week 2 — High Priority (22–79 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- Cross Border Transfer Risk — helpers/import-template.ts (4–12 hrs)
- Password Not Hashed — MULTIPLE (2–6 hrs)
- Audit Trail Partial (2–8 hrs)
- Retention Partial (2–8 hrs)
- No Error Handling In Auth — helpers/create-authenticated-customer.ts (1–3 hrs)
- Incomplete Deletion Coverage (2–8 hrs)
- Third Party Pii Sharing — MULTIPLE (3–10 hrs)
- Retention Missing — columns/helpers.ts (2–8 hrs)
- Retention Missing — schemas/address.ts (2–8 hrs)
- Deep Review Validated — helpers/create-authenticated-customer.ts (2–8 hrs)

Week 4 — Advisory (8–32 hours)

Address before next compliance review.

- Deep Review Validated — auth/medusa-config.js (2–8 hrs)
- Deep Review Validated — integration-tests/setup-env.js (2–8 hrs)
- Deep Review Validated — environment-helpers/setup-server.js (2–8 hrs)
- Deep Review Validated — components/cloud-auth-login.tsx (2–8 hrs)

Total estimated remediation effort: 42–158 hours (5.2–19.8 engineering days)

Details

Finding 1 of 25

[Section 8(1) — Security]

1. [HIGH] DEBUG_MODE_ENABLED — packages / medusa / src / commands / develop.ts:82**Evidence snippet:** NODE_ENV: "development",**Estimated fix time:** 4–16 hours

Debug mode is enabled. In production, this exposes interactive debuggers (Flask/Werkzeug), detailed stack traces, and internal system information to users. Flask debug mode specifically allows arbitrary code execution via the Werkzeug console. DPDP Section 8(1) requires appropriate security measures to protect personal data.

Risk:

Enabling debug mode in a production environment can expose sensitive system information and potentially allow arbitrary code execution, violating DPDP Section 8(1) security requirements.

Fix:

1. Ensure that the `BackendHmrFeatureFlag` and any other debug-related flags are disabled in the production deployment configuration.
2. Implement environment variable checks to disable development-specific features like HMR or detailed error reporting when `NODE\_ENV` is set to 'production'.
3. Review and remove any direct usage of debuggers or interactive consoles that are not intended for production environments.

DPDP:

Section 8(1) — Requires organizations to implement appropriate technical and organizational security measures to protect personal data.

Example:

```
if (process.env.NODE_ENV === 'production') { // Disable debug features
  FeatureFlag.disable(BackendHmrFeatureFlag.key); // Other production-specific configurations }
else { // Development configurations const isBackendHmrEnabled =
  FeatureFlag.isFeatureEnabled(BackendHmrFeatureFlag.key); // ... rest of development logic }
```

Finding 2 of 25

[Section 16 — Cross-Border Transfer]

2. [MEDIUM] CROSS_BORDER_TRANSFER_RISK — packages / admin / dashboard / src / routes / products / product-import / helpers / import-template.ts:4

Evidence snippet: ,coffee-mug-v3,Medusa Coffee Mug,,Every programmer's best friend.,published,https://medusa-public-images.s3.eu-west-1.amazonaws.com/coffee-mug.png,400,,,,,,,,,TRUE,,,One Size,,,FALSE,TRUE,,,,,,,,,

Estimated fix time: 4–12 hours

Foreign cloud storage/CDN detected with PII models in repo. Cross-border transfer risk.

Risk:

The finding indicates a risk of cross-border data transfer due to foreign cloud storage or CDNs being detected with PII models. This could lead to non-compliance with DPDP Section 16 if personal data is transferred without adequate safeguards.

Fix:

1. Audit all third-party services and CDNs used by the application to identify any that may be involved in cross-border data transfers.
2. If foreign cloud storage or CDNs are identified as handling personal data, implement data localization measures or ensure appropriate transfer mechanisms (e.g., Standard Contractual Clauses) are in place.
3. Review the application's data flow to minimize or eliminate the need for cross-border transfers of personal data where possible.

DPDP:

Section 16 — Governs the conditions under which personal data may be transferred outside of the jurisdiction.

Example:

```
import AWS from 'aws-sdk'; // Configure AWS SDK with region if necessary to control data residency
const s3 = new AWS.S3({ region: 'us-east-1', // Example: Specify a region to control data location });
// Function to upload data, ensuring it respects the configured region
async function uploadData(bucketName: string, key: string, data: Buffer) {
  await s3.putObject({
    Bucket: bucketName,
    Key: key,
    Body: data
  }).promise();
}
```

Finding 3 of 25

[Section 8(1) — Security]

3. [MEDIUM] PASSWORD_NOT_HASHED — MULTIPLE

Estimated fix time: 2–6 hours

Affected files (4):

- packages/cli/create-medusa-app/src/utls/create-db.ts
- packages/core/utls/src/common/handle-postgres-database-error.ts
- packages/core/utls/src/modules-sdk/load-module-database-config.ts
- packages/medusa/src/commands/db/create.ts

Model file has password field but no hash/bcrypt/argon/pbkdf2/scrypt/encrypt reference.

Detected in 4 files: create-db.ts, handle-postgres-database-error.ts, load-module-database-config.ts, create.ts

Risk:

Storing passwords without hashing or encryption violates DPDP Section 8(1) security requirements, exposing user accounts to compromise if the data is breached.

Fix:

1. Integrate a robust password hashing library (e.g., bcrypt, Argon2) into the user registration and password update flows.
2. Modify the user model or relevant service layer to automatically hash passwords before they are persisted to the database.
3. Ensure that all password comparisons during login are done against the hashed password stored in the database.

DPDP:

Section 8(1) — Requires organizations to implement appropriate technical and organizational security measures to protect personal data.

Example:

```
import bcrypt from 'bcrypt'; const saltRounds = 10; export async function
hashPassword(password: string): Promise<string> { return await bcrypt.hash(password,
saltRounds); } export async function comparePassword(password: string, hash: string):
Promise<boolean> { return await bcrypt.compare(password, hash); }
```


Finding 4 of 25

[Section 8(1) — Security]

4. [HIGH] HARDCODED_SECRET — MULTIPLE

Estimated fix time: 2–9 hours

Affected files (13):

- packages/core/js-sdk/src/admin/invite.ts
- packages/core/js-sdk/src/auth/index.ts
- www/utis/generated/oas-output/operations/admin/post_admin_invites_accept.ts
- www/utis/generated/oas-output/operations/admin/post_auth_[actor_type]_[auth_provider].ts
- www/utis/generated/oas-output/operations/admin/post_auth_[actor_type]_[auth_provider]_register.ts
- www/utis/generated/oas-output/operations/admin/post_auth_[actor_type]_[auth_provider]_update.ts
- ... and 7 more files

Hardcoded hardcoded_password detected. Detected in 13 files: invite.ts, index.ts, post_admin_invites_accept.ts, post_auth_[actor_type]_[auth_provider].ts and 9 more

Risk:

Hardcoding secrets like API keys, passwords, or encryption keys in the codebase creates a significant security risk, as they can be exposed if the code is compromised, violating DPDP Section 8(1).

Fix:

1. Replace all hardcoded secrets (e.g., JWT secrets, API keys, passwords) with references to environment variables or a dedicated secrets management system.
2. Ensure that sensitive configuration values are not committed to version control.
3. Implement a secure method for loading secrets at runtime, such as using libraries like `dotenv` for local development and a secure vault service in production.

DPDP:

Section 8(1) — Requires organizations to implement appropriate technical and organizational security measures to protect personal data.

Example:

```
import dotenv from 'dotenv'; dotenv.config(); const jwtSecret = process.env.JWT_SECRET; if (!jwtSecret) { throw new Error('JWT_SECRET environment variable is not set.')} // Use jwtSecret in your JWT generation/verification logic // Example: jwt.sign({ userId: '123' }, jwtSecret);
```

Finding 5 of 25

[Section 8(1) — Security]

5. [HIGH] PLAINTEXT_PII_IN_LOGS — packages / design-system / toolbox / src / commands / tokens / command.ts:181

Evidence snippet: logger.info(`Writing typography tokens to file`)

Estimated fix time: 1–3 hours

PII keyword appears in logging call. Personal data may be written to logs in plaintext.

Risk:

The script logs user emails during the super admin role assignment process. If these logs are not properly secured, this could expose personal data, violating the DPDP Act's security requirements.

Fix:

1. Modify the logging statements in `create-super-admin-role.ts` to avoid logging sensitive user information like email addresses. Consider logging only user IDs or a count instead.
2. Implement log sanitization or filtering at the application level to automatically remove or mask PII before it's written to logs.
3. Configure log rotation and access controls to ensure that logs are stored securely and only accessible by authorized personnel.

DPDP:

Section 8(1) — Security: Requires appropriate security measures to protect personal data, including preventing unauthorized access and disclosure.

Example:

```
async function assignSuperAdminRoleToUsers(container: any): Promise<void> { // ... other code
... logger.info(`■ Starting super admin role assignment script...`); // ... other code ...
users.forEach((user: User, index: number) => { // Instead of logging user.email, log only
user.id or a masked identifier logger.info(` ${index + 1}. User ID: ${user.id}`); }); // ...
other code ... }
```

Finding 6 of 25

[Section 8(1) — Security]

6. [HIGH] PLAINTEXT_PII_IN_LOGS — packages / medusa / src / migration-scripts / create-super-admin-role.ts:49**Evidence snippet:** `logger.info(` ${index + 1}. ${user.email || user.id} (${user.id})`)`**Estimated fix time:** 1–3 hours

PII keyword appears in logging call. Personal data may be written to logs in plaintext.

Risk:

The script logs user emails, which constitutes a partial audit trail as it lacks comprehensive logging of all access and changes. This makes it difficult to track data access and modifications, potentially hindering accountability and incident response.

Fix:

1. Implement a centralized audit logging mechanism that captures all significant events, including user authentication, data access, and modifications, across the application.
2. Define a clear audit log data model that includes timestamps, user identifiers, actions performed, and affected data.
3. Ensure that audit logs are stored securely and retained according to the organization's policy and regulatory requirements, separate from application logs.

DPDP:

Section 8(4) — Audit Trail: Requires maintaining a record of all operations performed on personal data, including access, modification, and deletion.

Example:

```
async function assignSuperAdminRoleToUsers(container: any): Promise<void> { const logger =
container.resolve(ContainerRegistrationKeys.LOGGER) const auditLogger =
container.resolve('auditLogger') // Assuming an audit logger service logger.info('■ Starting
super admin role assignment script...') auditLogger.audit('SCRIPT_START', { scriptName:
'create-super-admin-role' }) // ... existing code ... users.forEach((user: User, index:
number) => { auditLogger.audit('USER_PROCESSED', { userId: user.id, email: user.email })
logger.info(` ${index + 1}. User ID: ${user.id}`); // Keep for operational visibility if
needed }); }
```

Finding 7 of 25

[Section 8(4) — Audit Trail]

7. [MEDIUM] AUDIT_TRAIL_PARTIAL — N / A

Estimated fix time: 2–8 hours

Partial audit trail detected. Found: change tracking. Missing: access logging, audit model.

Risk:

The codebase lacks a defined retention policy for personal data, including customer email addresses and names. This means data may be stored indefinitely, increasing the risk associated with data breaches and non-compliance with data minimization principles.

Fix:

1. Define a data retention policy that specifies how long different types of personal data should be stored, aligning with business needs and DPDP requirements.
2. Implement mechanisms to automatically delete or anonymize personal data once it has exceeded its retention period. This could involve scheduled jobs or event-driven processes.
3. Document the retention policy and the implementation details, making it accessible to relevant stakeholders and for audit purposes.

DPDP:

Section 8(3) — Retention: Requires that personal data is not kept for longer than necessary for the purposes for which it is processed.

Example:

```
import { CreateCustomerDTO } from "@medusajs/types" // Assume a configuration service or
constants define retention periods const RETENTION_PERIOD_CUSTOMER_EMAIL = 365; // days const
RETENTION_PERIOD_CUSTOMER_NAME = 730; // days export const createAuthenticatedCustomer = async
( api: any, storeHeaders: Record<any, any>, customerData: Partial<CreateCustomerDTO> = {} ) =>
{ const email = customerData.email ?? "tony@start.com" // ... other code ... const { data: {
customer } } = await api.post( "/store/customers", { email, first_name: "John", last_name:
"Doe", metadata: { created_at: new Date().toISOString() }, // Add creation timestamp for
retention tracking ...customerData, }, // ... headers ... ) // ... rest of the function ... }
```

Finding 8 of 25

[Section 8(3) — Retention]

8. [MEDIUM] RETENTION_PARTIAL — N / A

Estimated fix time: 2–8 hours

Retention signals found in 3 model(s) but 2 PII model(s) have no retention policy. Uncovered: helpers.ts, address.ts.

Risk:

The codebase does not explicitly define retention policies for PII models such as customer addresses. This absence means that personal data might be retained indefinitely, increasing the risk of unauthorized access or misuse and violating data minimization principles.

Fix:

1. Identify all data models that contain PII, including address information.
2. Define specific retention periods for each PII data model based on legal requirements and business necessity.
3. Implement automated processes for data deletion or anonymization once the retention period expires for PII fields within these models.

DPDP:

Section 8(3) — Retention: Requires that personal data is not kept for longer than necessary for the purposes for which it is processed.

Example:

```
import { CreateCustomerDTO } from "@medusajs/types" // Assume a configuration service or
constants define retention periods const RETENTION_PERIOD_ADDRESS = 730; // days export const
createAuthenticatedCustomer = async ( api: any, storeHeaders: Record<any, any>, customerData:
Partial<CreateCustomerDTO> = {} ) => { const email = customerData.email ?? "tony@start.com" //
... other code ... const { data: { customer } } = await api.post( "/store/customers", { email,
first_name: "John", last_name: "Doe", address: { // Assuming address is part of customerData
or a separate field // Add a timestamp or flag for retention tracking if needed
...customerData.address, }, metadata: { created_at: new Date().toISOString() },
...customerData, }, // ... headers ... ) // ... rest of the function ... }
```

Finding 9 of 25

[Section 8(6) — Breach]

9. [MEDIUM] NO_ERROR_HANDLING_IN_AUTH — integration-tests / modules / helpers / create-authenticated-customer.ts

Estimated fix time: 1–3 hours

Auth routes have no try/catch or try/except blocks. Errors may go unlogged.

Risk:

Authentication routes lack error handling, which can lead to unlogged errors and potential security vulnerabilities if exceptions are not properly managed. This failure to log breaches could impede the ability to respond effectively to security incidents as required by DPDP Section 8(6).

Fix:

1. Add try/catch blocks around the `api.post` calls in `createAuthenticatedCustomer` function in `integration-tests/modules/helpers/create-authenticated-customer.ts` to handle potential errors during authentication and customer creation.
2. Inside the catch blocks, log the error details using a suitable logging mechanism (e.g., `console.error` or a dedicated logger) to ensure errors are recorded.
3. Consider returning a specific error response or re-throwing a custom error to indicate failure to the calling function.

DPDP:

Section 8(6) — Breach

Example:

```
import { CreateCustomerDTO } from "@medusajs/types" export const createAuthenticatedCustomer =
async ( api: any, storeHeaders: Record<any, any>, customerData: Partial<CreateCustomerDTO> =
{} ) => { const email = customerData.email ?? "tony@start.com" try { const signup = await
api.post(REDACTED_POSSIBLE_SECRET, { email, password = "REDACTED_CREDENTIAL", }) // ... rest
of the function with try/catch for other api calls } catch (error) { console.error("Error
during authentication: ", error) throw error // or return an error response } }
```

Finding 10 of 25

[Section 8 — Data Principal Rights]

10. [MEDIUM] INCOMPLETE_DELETION_COVERAGE — N / A**Estimated fix time:** 2–8 hours

Deletion mechanism detected but may not cover all PII storage. Uncovered files: integration-tests/modules/src/modules/brand/models/brand.ts, packages/admin/dashboard/src/lib/table/entity-defaults.ts, packages/cli/http-types-generator/src/core/schema-extractor.ts, packages/core/framework/src/database/index.ts, packages/core/framework/src/database/pg-connection-loader.ts. DPDP Section 8 requires complete erasure of all personal data across all systems when a user requests deletion.

Risk:

The absence of a comprehensive data deletion mechanism means that personal data may persist in the system even after a user requests its removal, violating DPDP Section 8 requirements for erasure. This can lead to continued unauthorized processing and potential non-compliance with data subject rights.

Fix:

1. Implement a data deletion service that identifies and removes PII from all relevant data stores, including databases, logs, and any other storage locations.
2. Ensure the deletion process is triggered by a clear user request or an administrative action.
3. For each identified PII storage location (e.g., `integration-tests/modules/src/modules/brand/models/brand.ts`, `packages/cli/http-types-generator/src/core/schema-extractor.ts`), create specific deletion logic.
4. Test the deletion mechanism thoroughly to confirm complete erasure of data associated with a specific data principal.

DPDP:

Section 8 — Data Principal Rights

Example:

```
import { CustomerRepository } from "@medusajs/medusa/dist/repositories/customer" import {
OrderRepository } from "@medusajs/medusa/dist/repositories/order" // ... import other
repositories export async function deleteUserData(customerId: string) { const customerRepo =
app.resolve(CustomerRepository) // ... get other repositories await customerRepo.delete({ id:
customerId }) // ... call delete methods for other relevant repositories console.log(`User
data for customer ${customerId} deleted successfully.`)
```

Finding 11 of 25

[Section 5 — Notice]

11. [MEDIUM] THIRD_PARTY_PII_SHARING — MULTIPLE:245**Evidence snippet:** const shippingAddress = order.shipping_address ?? { id: undefined }**Estimated fix time:** 3–10 hours**Affected files (3):**

- packages/core/core-flows/src/order/workflows/create-fulfillment.ts
- packages/core/core-flows/src/order/workflows/create-order.ts
- packages/design-system/ui/src/components/date-segment/date-segment.tsx

Personal data present in file that imports third-party SDK. Detected in 3 files: create-fulfillment.ts, create-order.ts, date-segment.tsx

Risk:

Personal data, such as addresses, is being processed in a workflow that may involve third-party integrations without clear controls or notice mechanisms. This could lead to unauthorized sharing of PII, violating DPDP Section 5 requirements for transparency and notice.

Fix:

1. Review the `create-order.ts` workflow and identify all third-party services or SDKs that interact with the processed personal data (e.g., `shipping\_address`, `billing\_address`).
2. For each identified third-party interaction, ensure a clear notice is provided to the data principal about the data sharing, as per DPDP Section 5.
3. Implement mechanisms to obtain explicit consent or rely on a legitimate basis for sharing PII with third parties, documenting this basis.
4. If no third-party sharing is intended, add explicit code or configuration to prevent such sharing and document this decision.

DPDP:

Section 5 — Notice

Example:

```
import { Customer } from "@medusajs/medusa" import { OrderService } from "@medusajs/medusa"
import { MedusaContainer } from "@medusajs/modules-sdk" // Assume 'container' is an instance
of MedusaContainer const orderService = container.resolve(OrderService) const customer = await
orderService.retrieve(customerId, { select: ["email", "first_name", "last_name"] }) //
Example: If sharing with a shipping provider: // await
shippingProviderService.processAddress(customer.shipping_address) // Ensure notice is given to
the customer about this sharing.
```

Finding 12 of 25

[Section 8(3) — Retention]

12. [MEDIUM] RETENTION_MISSING — packages / medusa / src / api / admin / views / [entity] / columns / helpers.ts**Estimated fix time:** 2–8 hours

Model/schema file contains PII fields but no retention or expiry signals detected. Data may be retained indefinitely.

Risk:

Manual review required. Rule 'RETENTION_MISSING' fired on packages/medusa/src/api/admin/views/[entity]/columns/helpers.ts — see rule description for details.

Fix:

1. Review the finding description above and apply fixes per your compliance process.

DPDP:

Section 8(3) — Retention

Finding 13 of 25

[Section 8(3) — Retention]

13. [MEDIUM] RETENTION_MISSING — packages / plugins / draft-order / src / admin / lib / schemas / address.ts:5**Evidence snippet:** The schema definition itself does not contain any explicit fields or configurations related to data retention periods or expiry.**Estimated fix time:** 2–8 hours

Model/schema file contains PII fields but no retention or expiry signals detected. Data may be retained indefinitely.

Risk:

The address schema contains PII fields without any defined retention or expiry periods, meaning this data could be stored indefinitely, violating data minimization principles.

Fix:

1. Add a `retention\_period\_days` field to the `addressSchema` using `z.number().positive()` and document its purpose.
2. Implement a data lifecycle management process that automatically purges or anonymizes address data exceeding the defined retention period.
3. Update any data processing or storage logic that uses this schema to respect the new retention period.

DPDP:

Section 8(3) — Data minimization and storage limitation require that personal data is not kept longer than necessary for the purposes for which it is processed.

Example:

```
import { z } from "zod" export const addressSchema = z.object({ country_code:
z.string().min(1), first_name: z.string().min(1), last_name: z.string().min(1), address_1:
z.string().min(1), address_2: z.string().nullish(), company: z.string().nullish(), city:
z.string().min(1), province: z.string().nullish(), postal_code: z.string().min(1), phone:
z.string().nullish(), retention_period_days: z.number().positive().optional(), // Added
retention field })
```

Finding 14 of 25

[Section 7 — Legitimate Use]

14. [INFO] INTERNAL_ROUTE_PII_ACCESS — packages / core / js-sdk / src / admin / stock-location.ts

Internal dashboard/admin route accesses PII. Verify: (1) access is restricted to authorised staff only via role-based middleware, (2) a Section 7 legitimate use basis is documented in your compliance register.

Finding 15 of 25

[Section 7 — Legitimate Use]

15. [INFO] INTERNAL_ROUTE_PII_ACCESS — packages / cli / medusa-cli / src / commands / new.ts

Internal file (new.ts) accesses PII. Verify access is restricted to authorised contexts and PII is not exposed to logs or error responses.

Finding 16 of 25

[Section 7 — Legitimate Use]

16. [INFO] INTERNAL_ROUTE_PII_ACCESS — integration-tests / modules / helpers / create-authenticated-customer.ts

Internal service/utility accesses PII. Verify: (1) this file is only called from authorised contexts, (2) PII is not inadvertently exposed to logs or error messages.

Finding 17 of 25

[Section 6(6) — Consent Withdrawal]

17. [PASS] CONSENT_WITHDRAWAL_PRESENT — packages / admin / dashboard / src / routes / workflow-executions / workflow-execution-detail / components / workflow-execution-timeline-section / workflow-execution-timeline-section.tsx

Consent withdrawal mechanism detected. Found in: packages/admin/dashboard/src/routes/workflow-executions/workflow-execution-detail/components/workflow-execution-timeline-section/workflow-execution-timeline-section.tsx (line 105)

Finding 18 of 25

[Section 8(6) — Breach]

18. [PASS] BREACH_INDICATORS_ADEQUATE — N / A

Logging and/or breach alerting present.

Finding 19 of 25

[Section 8(1)]

19. [HIGH] DEEP_REVIEW_VALIDATED — integration-tests / http / __fixtures__ / auth / medusa-config.js

Estimated fix time: 2–8 hours

The file `integration-tests/http/\_\_fixtures\_\_/auth/medusa-config.js` uses a hardcoded passphrase 'secret' for generating RSA key pairs used in JWT authentication.

Finding 20 of 25

[Section 8(1)]

20. [MEDIUM] DEEP_REVIEW_VALIDATED — integration-tests / modules / helpers / create-authenticated-customer.ts

Estimated fix time: 2–8 hours

The `createAuthenticatedCustomer` function in `integration-tests/modules/helpers/create-authenticated-customer.ts` uses a hardcoded placeholder 'REDACTED_CREDENTIAL' for the password.

Finding 21 of 25

[Section 8(1)]

21. [LOW] DEEP_REVIEW_VALIDATED — integration-tests / http / __fixtures__ / auth / medusa-config.js

Estimated fix time: 2–8 hours

In `integration-tests/http/\_\_fixtures\_\_/auth/medusa-config.js`, the JWT configuration is present in both `projectConfig.http` and `modules[Modules.USER].options`.

Finding 22 of 25

[Section 8(1)]

22. [HIGH] DEEP_REVIEW_VALIDATED — integration-tests / api / medusa-config.js

Estimated fix time: 2–8 hours

The file `integration-tests/api/medusa-config.js` contains hardcoded secrets like `jwtSecret: "test"` and `cookieSecret: "test"`.

Finding 23 of 25

[Section 8(1)]

23. [LOW] DEEP_REVIEW_VALIDATED — integration-tests / setup-env.js**Estimated fix time:** 2–8 hours

In `integration-tests/setup-env.js`, the `process.env.DB\_TEMP\_NAME` is set based on `JEST\_WORKER\_ID` and `CHUNK` environment variables. The `integration-tests/environment-helpers/setup-server.js` file also sets `REDIS\_URL` based on `JEST\_WORKER\_ID`.

Finding 24 of 25

[Section 8(1)]

24. [LOW] DEEP_REVIEW_VALIDATED — integration-tests / environment-helpers / setup-server.js**Estimated fix time:** 2–8 hours

The `integration-tests/environment-helpers/setup-server.js` file merges `process.env` with provided `env` object and hardcoded values like `JWT\_SECRET` and `COOKIE\_SECRET`.

Finding 25 of 25

[Section 8(1)]

25. [LOW] DEEP_REVIEW_VALIDATED — packages / admin / dashboard / src / routes / login / components / cloud-auth-login.tsx**Estimated fix time:** 2–8 hours

In packages/admin/dashboard/src/routes/login/components/cloud-auth-login.tsx, the `decodeToken` function is used to parse the JWT. The decoded token's `actor\_id` and `user\_metadata` are then used directly.

AI-Identified Compliance Gaps

The following gaps were identified by AI analysis and cross-validated. These are advisory observations, not verified rule violations. Human review is required before acting on these findings.

[AI-INFERRED] No explicit age verification for account creation

Confidence: 45%

Section 9 — Children

The repository context does not mention any specific mechanisms for age verification during user account creation, which is a requirement for processing children's personal data under Section 9.

Recommendation: Implement an age verification step during the user registration process. This could involve a simple age gate or more robust verification methods if deemed necessary based on risk assessment.

[AI-INFERRED] Potential lack of comprehensive data access endpoint

Confidence: 40%

Section 8 — Right to Access

While the rule engine coverage map indicates 'deletion_checked' is true, there's no explicit mention of a check for a 'right_to_access_checked' or similar. This suggests that the ability for users to download all their personal data (Section 8) might not have been explicitly verified.

Recommendation: Develop and verify an endpoint that allows users to request and download all personal data held by the system. Ensure this endpoint is accessible and functional.

[AI-INFERRED] No exposed Data Protection Officer contact

Confidence: 35%

Section 10 — DPO

The repository context and rule engine coverage do not indicate any exposure of a Data Protection Officer (DPO) contact information. Section 10 of DPDP Act implies that such contact should be made available.

Recommendation: Add contact information for the Data Protection Officer to a publicly accessible location, such as a privacy policy or a dedicated contact page.

Deep Compliance Review

Overall Risk: HIGH

Significant security vulnerabilities exist across multiple layers, particularly in authentication and configuration, with hardcoded secrets and plaintext credentials posing a direct risk to sensitive data. Inconsistent handling of PII and potential for over-fetching in workflows also present compliance gaps. Addressing these issues is critical to prevent unauthorized access and data breaches.

Estimated remediation: 17 engineering day(s)

Most Critical Action: Remove all hardcoded secrets and passphrases from integration test configurations and code, replacing them with secure environment variable injection or a dedicated secrets management solution.

Layer Breakdown

Data Models & Storage — 0 finding(s). The reviewed code defines data models and structures for a data grid component. No direct DPDP compliance issues were identified in the provided snippets. The code appears to focus on data representation and manipulation within the UI layer, rather than direct handling or storage of sensitive personal information in a way that would violate DPDP principles. However, it's crucial that the broader application context ensures that any PII processed through these models is handled according to DPDP requirements, including consent, purpose limitation, and security.

Authentication & Session — 3 finding(s). The authentication layer relies heavily on JWT for session management. The configuration of JWT keys, particularly in test environments, needs careful attention to avoid hardcoded secrets and ensure secure generation and storage of keys. The use of `crypto.generateKeyPairSync` with a passphrase is a good practice, but the passphrase itself must be managed securely.`

Third-Party & PII Flows — 2 finding(s). The workflows reviewed focus on order and claim management. While no direct PII handling is evident, the broad data fetching in some queries could inadvertently include or be linked to sensitive information, posing a risk under data minimization principles. Explicitly limiting fetched fields to only what's necessary is recommended.

Configuration & Secrets — 3 finding(s). The configuration layer for integration tests exhibits several weaknesses related to secret management and environment variable handling. Hardcoded secrets in ``medusa-config.js`` are a significant risk. Additionally, the reliance on worker IDs for generating database names and the potential for sensitive data in passed environment variables indicate architectural patterns that could lead to data exposure, even in test environments. These issues require attention to ensure robust data protection practices are followed.

API Routes & Controllers — 3 finding(s). The API Routes & Controllers layer primarily handles user authentication and password reset flows. Key areas of concern include the client-side handling of authentication tokens and password reset tokens. While JWTs are used, their decoding and direct use in the client can present minor risks if not managed carefully. The implementation of ``react-jwt`` for decoding tokens is noted, and recommendations focus on minimizing client-side exposure of sensitive token data and ensuring robust backend validation.